DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

Date: 10/11/2025	**CIE - 1**	**Max. Marks: 60**
Semester: V	**UG**	**Duration: 2 Hour**
Course Title: ADVANCED ALGORITHMS		**Course Code: CS355TBB**

SCHEME AND SOLUTION

Sl. No.	PART A	M	BT	CO
1.	Lower bound	1	1	1
2.	$O(n^2)$	2	2	1
3.	The O(log n)	2	3	1
4.	O(n + k)	1	1	1
5.	O(1)	1	2	2
6.	Accounting method	1	1	3
7.	Ω(n log n)	1	2	2
8.	2^n	1	2	3
	PART B			
1a.	The Recursion Tree Method: • Visualizing per-level work in recursive algorithms. • Understanding how cost accumulates across recursion levels. • Identifying dominant levels contributing to overall complexity. f(1)=1·log1=0, but leaves contribute base T(1)=Θ(1) • Number of subproblems: a=2 • Subproblem size: n/b=n/2 • Work per level: f(n)= nlogn Total Cost: **T(n)=Θ(n(logn)2)**	2+4	3	1
1b.	Functions: f1(n) = nlogn, f2(n)=$n^{1.5}$, f3(n)=2n, f4(n)=n^2 f1(n)=o(f2(n)), f2(n)=o(f4(n)), f4(n)=o(f3(n))). Equivalently: **nlogn∈o(n1.5),n1.5∈o(n2),n2∈o(2n).**	2+2	3	2
2.	Counting Sort: Step 1 — Initialize Counting Array Step 2 — Count Each Element Step 3 — Compute Cumulative Count Step 4 — Build Output Array **Algorithm:** COUNTING-SORT(A, B, k) 1. Let C[0…k] be a new array 2. for i ← 0 to k C[i] ← 0 3. for j ← 1 to length[A]	4+4+2	3	4

	C[A[j]] ← C[A[j]] + 1 4. for i ← 1 to k C[i] ← C[i] + C[i – 1] 5. for j ← length[A] downto 1 B[C[A[j]]] ← A[j] C[A[j]] ← C[A[j]] – 1 Time Complexity: **O(n+k)**			
3.	Accounting Method: Under the charging scheme (3 credits per operation), the stored credits always cover the cost of any resizing (doubling or halving). Amortized cost per insertion = O(1) Amortized cost per deletion = O(1) More precisely, with the scheme above amortized cost ≤ **3** time units per operation, that will be **Θ(1)**.	5+5	2	2
4a.	a) T(n)=2T(n/2)+n • a=2, b=2⇒p= log2=1. • f(n)=n=$\Theta(n^1)$=$\Theta(n^p)$ This matches Case 2 with k=0. **T(n)=Θ(nlogn).** (b) T(n)=3T(n/4)+ n2 a=3, b=4⇒p=$\log_4 3$. ≈0.792. f(n)=n2. Compare exponents: 2> p, so f(n)=Ω(np+ϵ) for ϵ=2−p>0 Case 3 applies , **T(n)=Θ(n2).**	3+3	3	1
4b.	Any comparison-based sorting algorithm needs at least Ω(n log n) comparisons in the worst case. Implication: • Algorithms like Merge Sort and Heap Sort (which take O(nlogn)) are asymptotically optimal. • To achieve faster sorting (like O(n)), algorithms must be non-comparison-based (e.g., Counting Sort, Radix Sort).	4	2	3
5.	**Radix Sort:** All numbers are 3-digit base-10 integers → we will have 3 passes: 1. Pass 1: Sort by unit's place (1s digit) 2. Pass 2: Sort by ten's place (10s digit) 3. Pass 3: Sort by hundred's place (100s digit) **Algorithm** RADIX-SORT(A, d) // A: list of numbers, d: number of digits 1. for i ← 1 to d do 2. use a stable sort (e.g., Counting Sort) 3. to sort array A according to the i-th least significant digit Time Complexity: **O(dn)**	3+4+3	3	1

BT-Blooms Taxonomy, CO-Course Outcomes, M-Marks

Marks Distribution	Particulars		CO1	CO2	CO3	CO4	L1	L2	L3	L4	L5	L6
	Test	Max Marks	21	13	16	10	4	19	31	--	6	-

Course Outcomes: After completing the course, the students will be able to: -	
CO 1	Analyze various algorithms for their time and space complexity.
CO 2	Demonstrate a familiarity with major algorithms and data structures
CO 3	Apply appropriate design techniques for solving real world problems.
CO 4	Design and implement solutions using appropriate mathematical techniques.